



regmap



regmap

- ▶ has its roots in ASoC (ALSA)
- ▶ can use I2C, SPI and MMIO (also SPMI)
- ▶ actually abstracts the underlying bus
- ▶ can handle locking when necessary
- ▶ can cache registers
- ▶ can handle endianness conversion
- ▶ can handle IRQ chips and IRQs
- ▶ can check register ranges
- ▶ handles read only, write only, volatile, precious registers
- ▶ handles register pages
- ▶ API is defined in [include/linux/regmap.h](#)
- ▶ implemented in [drivers/base/regmap/](#)



regmap: creation

- ▶

```
#define regmap_init(dev, bus, bus_context, config)
    __regmap_lockdep_wrapper(__regmap_init, #config,
                             dev, bus, bus_context, config)
```
- ▶

```
#define regmap_init_i2c(i2c, config)
    __regmap_lockdep_wrapper(__regmap_init_i2c, #config,
                             i2c, config)
```
- ▶

```
#define regmap_init_spi(dev, config)
    __regmap_lockdep_wrapper(__regmap_init_spi, #config,
                             dev, config)
```
- ▶ Also `devm_` versions
- ▶ and `_clk` versions, preparing, enabling and disabling clocks when necessary



regmap: config

include/linux/regmap.h

```
struct regmap_config {
[...]  
    int reg_bits;  
    int reg_stride;  
[...]  
    bool (*writeable_reg)(struct device *dev, unsigned int reg);  
    bool (*readable_reg)(struct device *dev, unsigned int reg);  
    bool (*volatile_reg)(struct device *dev, unsigned int reg);  
    bool (*precious_reg)(struct device *dev, unsigned int reg);  
[...]  
    int (*reg_read)(void *context, unsigned int reg, unsigned int *val);  
    int (*reg_write)(void *context, unsigned int reg, unsigned int val);  
    int (*reg_update_bits)(void *context, unsigned int reg,  
                           unsigned int mask, unsigned int val);  
[...]  
    const struct reg_default *reg_defaults;  
    unsigned int num_reg_defaults;  
[...]  
};
```



regmap: config

- ▶ `reg_bits` Number of bits in a register address, mandatory.
- ▶ `reg_stride` The register address stride. Valid register addresses are a multiple of this value. If set to 0, a value of 1 will be used.
- ▶ `writeable_reg`, `readable_reg`, `volatile_reg`, `precious_reg`: Optional callbacks returning true if the register is writeable, readable, volatile or precious. volatile registers won't be cached. precious registers will not be read unless the driver explicitly calls a read function. There are also tables in the `struct regmap_config` for the same purpose.
- ▶ `reg_read`, `reg_write`, `reg_update_bits`: Optional callbacks that if filled will be used to perform accesses. `reg_update_bits` should only be provided if specific locking is required.
- ▶ `reg_defaults`: Power on reset values for registers (for use with register cache support).
- ▶ `num_reg_defaults`: Number of elements in `reg_defaults`.



regmap: access

- ▶ `int regmap_read(struct regmap *map, unsigned int reg, unsigned int *val);`
- ▶ `int regmap_write(struct regmap *map, unsigned int reg, unsigned int val);`
- ▶ `static inline int regmap_update_bits(struct regmap *map, unsigned int reg,
 unsigned int mask, unsigned int val)`
- ▶ `#define regmap_read_poll_timeout(map, addr, val, cond, sleep_us, timeout_us)`
- ▶ `int regmap_test_bits(struct regmap *map, unsigned int reg, unsigned int bits);`
- ▶ `static inline int regmap_update_bits_check(struct regmap *map, unsigned int reg,
 unsigned int mask, unsigned int val,
 bool *change)`



regmap: cache management

- ▶ `int regcache_sync(struct regmap *map);`
- ▶ `int regcache_sync_region(struct regmap *map, unsigned int min, unsigned int max);`
- ▶ `int regcache_drop_region(struct regmap *map, unsigned int min, unsigned int max);`
- ▶ `void regcache_cache_only(struct regmap *map, bool enable);`
- ▶ `void regcache_cache_bypass(struct regmap *map, bool enable);`
- ▶ `void regcache_mark_dirty(struct regmap *map);`



regmap: example

sound/soc/codecs/max9877.c

```
static const struct regmap_config max9877_regmap = {
    .reg_bits = 8,
    .val_bits = 8,

    .reg_defaults = max9877_regs,
    .num_reg_defaults = ARRAY_SIZE(max9877_regs),
    .cache_type = REGCACHE_RBTREE,
};

static int max9877_i2c_probe(struct i2c_client *client)
{
    struct regmap *regmap;
    int i;

    regmap = devm_regmap_init_i2c(client, &max9877_regmap);
    if (IS_ERR(regmap))
        return PTR_ERR(regmap);

    /* Ensure the device is in reset state */
    for (i = 0; i < ARRAY_SIZE(max9877_regs); i++)
        regmap_write(regmap, max9877_regs[i].reg, max9877_regs[i].def);

    return devm_snd_soc_register_component(&client->dev,
```




regmap: example

```
&max9877_component_driver, NULL, 0);
```

```
}
```



regmap: i2c and spi device example

sound/soc/codecs/adau1372.c

```
const struct regmap_config adau1372_regmap_config = {
    .val_bits = 8,
    .reg_bits = 16,
    .max_register = 0x4d,

    .reg_defaults = adau1372_reg_defaults,
    .num_reg_defaults = ARRAY_SIZE(adau1372_reg_defaults),
    .volatile_reg = adau1372_volatile_register,
    .cache_type = REGCACHE_RBTREE,
}; EXPORT_SYMBOL_GPL(adau1372_regmap_config);
```

sound/soc/codecs/adau1372-i2c.c

```
static int adau1372_i2c_probe(struct i2c_client *client)
{
    return adau1372_probe(&client->dev,
        devm_regmap_init_i2c(client, &adau1372_regmap_config), NULL);
}
```



regmap: i2c and spi device example

sound/soc/codecs/adau1372-spi.c

```
static int adau1372_spi_probe(struct spi_device *spi)
{
    struct regmap_config config;

    config = adau1372_regmap_config;
    config.read_flag_mask = 0x1;

    return adau1372_probe(&spi->dev,
        devm_regmap_init_spi(spi, &config), adau1372_spi_switch_mode);
}
```



regmap: ASoC components

- ▶ `snd_soc_component` regmap accessors also exist, they are available either implicitly as the component core calls `dev_get_regmap(component->dev, NULL)` to retrieve or create a regmap for the device or explicitly by calling `snd_soc_component_init_regmap()`

`include/sound/soc-component.h`

```
/* component IO */
unsigned int snd_soc_component_read(struct snd_soc_component *component,
                                   unsigned int reg); int snd_soc_component_write(struct
snd_soc_component *component,
                                   unsigned int reg, unsigned int val); int snd_soc_component_update_bits(struct
snd_soc_component *component,
                                   unsigned int reg, unsigned int mask,
                                   unsigned int val);

[...]
int snd_soc_component_test_bits(struct snd_soc_component *component,
                               unsigned int reg, unsigned int mask,
                               unsigned int value);
```